

Software Requirements Specification (SRS)

1. Introduction

****System Name:** EVOLVE - Software Change Request Management System**

****Purpose:****

EVOLVE is a web-based application designed to manage the lifecycle of software change requests (CRs). It helps engineering teams track requests, analyze the ripple effect of proposed changes across the codebase, and manage the workflow of implementing those changes.

2. Overall Description

The system provides a reactive Single Page Application (SPA) interface where developers and managers can submit change requests, view them on a Kanban board, analyze the risk and impact of the changes, and trace code-level modifications.

3. Functional Requirements

3.1 Change Request Management

* ****FR-1:**** Users must be able to create a new Change Request (CR) specifying a Title, Description, Priority (Low, Medium, High, Critical), Category (Feature, Bug Fix, Refactoring, Optimization), and Affected Components.

* ****FR-2:**** The system must generate a unique ID for each CR.

* ****FR-3:**** The system must default new CRs to a "Draft" status.

3.2 Kanban Board Workflow

* ****FR-4:**** The system must display CRs on a Kanban board with predefined columns: Draft, Submitted, Impact Analysis, Approved, In Progress, Testing, Merged.

* ****FR-5:**** Users must be able to change the status of a CR by dragging and dropping the CR card between columns.

3.3 Impact Analysis

* **FR-6:** The system must calculate an evolution risk score (0-100) and risk level (Low, Medium, High, Critical) for components affected by a CR.

* **FR-7:** The impact calculation must propagate through component dependencies, calculating distance, complexity, and churn rate to assess transitive impacts.

* **FR-8:** The system must visualize the change impact propagation (direct and transitive impacts).

3.4 Traceability & Code Diffs

* **FR-9:** The system must generate simulated code diffs for components affected by a CR.

* **FR-10:** Users must be able to view these code diffs for traceability purposes.

4. Non-Functional Requirements

4.1 User Interface

* **NFR-1:** The UI must be highly responsive and update reactively to state changes without full page reloads.

* **NFR-2:** The application must utilize a modern design system with a specific font stack and CSS variables.

4.2 Performance

* **NFR-3:** Impact analysis calculations should execute in real-time (under 100ms) for typical codebase sizes to maintain UI fluidity.

4.3 Architecture constraints

* **NFR-4:** The application must be built using Vanilla JavaScript and CSS, without relying on heavy front-end frameworks (like React or Angular).